



**Tshwane University
of Technology**

We empower people

**INSTRUCTIONS TO
CANDIDATES**

1. All examination rules stated by the Tshwane University of Technology apply.
2. Ensure a single final version of your source code is handed in as requested.
3. State all necessary assumptions clearly in code commentary where needed.

MARKS: 50

PAGES: 11 (incl. cover)

EXAMINER:

Mr A.J. Smith

Mr M. Chauke

Prof J.A. Jordaan

MODERATOR:

Mr D Engelbrecht

TIME:

90 Minutes

15 Minutes extra time in the event
of computer problems

**FACULTY OF
ENGINEERING AND
THE BUILT ENVIRONMENT**

**DEPARTMENT OF
ELECTRICAL ENGINEERING**

**ES216BB
ENGINEERING SOFTWARE DESIGN B**

EVALUATION 1 A

AUGUST 2026

EVALUATION INSTRUCTIONS

1. **Plagiarism:** Submit only original work. Similarity software will verify the authenticity of all submissions.
2. **Permitted Tools:** Use only CodeBlocks and Google Chrome to access the evaluation, view the paper, and upload your submission. Email, other online resources, memory sticks and unauthorised tools are prohibited.
3. **File Submission:** Rename the supplied source template to <student number>.cpp. Do not add a name, surname or other text to the file name.
4. **Uploading Instructions:** ONLY SUBMIT YOUR SOURCE FILE (.cpp) through the designated upload link. The latest submission is retained.
5. **Evaluation Scope:** This assessment covers Engineering Software Design B Units 1 to 3.
6. **Programming Language:** Construct the required program in C++11 and follow structured programming principles.
7. **Editing and Requirements:** Meet all specified requirements and use the attached appendices as required.
8. **Evaluation Requirements:**
 - a. Remember to save your work on the PC D: drive and save regularly throughout the evaluation.
 - b. Do not modify the given libraries, type declarations, prototypes, menus or comments in the supplied .cpp template.
 - c. Complete every required function in the comment blocks and use the exact names and parameters supplied in the question paper and template.
 - d. The supplied input file is **inverter_tests.txt**.

C++ FILE CODE EXPLANATION

The supplied C++ file sets up a menu-driven Solar Inverter Diagnostic System. It imports records from *inverter_tests.txt*, stores them in a dynamically allocated array, and exports the required report to *inverter_fault_alerts.txt*.

The system includes functionalities for:

- **Load:** Diagnostic records from a text file.
- **Calculate:** Inverter conversion efficiency.
- **Filter:** Warning and fault records.
- **Export:** Fault alerts for maintenance.

The program uses a structure named *InverterTest* with direct data members, a static reference record accessed with the dot operator, and a dynamic array accessed through a pointer and arrow operator.

The required type declarations and functions below must be implemented in the appropriate comment blocks in the supplied .cpp template file.

Use the exact function names and parameters from the supplied template.

STRUCTURE DEFINITION EXPLANATION

The InverterTest structure and supporting types define the properties and operations required by this engineering system:

- **Structure and type requirements:**
 - **inverterID** (int): unique inverter identifier.
 - **dcInput** and **acOutput** (float): power readings.
 - **stateCode** (char): P, W or F status code.
 - **diagnosticValue** (float): direct diagnostic reading or fault code.
 - **tests pointer** and **arraySize**: dynamic diagnostic data.
- **Required functions:**
 - **InitialiseInverterTest()**: initialise one record.
 - **TextFileLineCount()**: validate and count file records.
 - **ReadFileAndPopulate()**: create/reload dynamic data.
 - **CalculateEfficiency()**: perform the scenario calculation.
 - **DisplaySystemName()**: display scenario information.
 - **DisplayInverterTests()**: complete the rotated Unit 1 technique.
 - Report and cleanup functions: export results and release dynamic memory.

FUNCTION EXPLANATIONS

1. InitialisInverterTest Function

void InverterTest::InitialisInverterTest(int, float, float, char, float)

Purpose: Initialises one inverter-test structure using direct data members.

Parameters: id, power readings, state code and diagnostic value.

Returns: No value.

2. TextFileLineCount Function

int TextFileLineCount(string fileName)

Purpose: Checks that the text file is available and counts non-empty records.

Parameters: fileName: input text-file name.

Returns: Number of valid records, or zero.

3. ReadFileAndPopulate Function

void ReadFileAndPopulate(string, InverterTest **, int *)

Purpose: Releases prior data, allocates the dynamic array and loads every record with arrow access.

Parameters: file name, dynamic-array pointer and size pointer.

Returns: No value; updates the caller data.

4. CalculateEfficiency Function

inline float CalculateEfficiency(InverterTest test)

Purpose: Calculates AC output as a percentage of DC input and handles zero input safely.

Parameters: test: one inverter-test structure.

Returns: Efficiency percentage.

5. DisplaySystemName Function

void DisplaySystemName()

Purpose: Uses a local name and ::systemName to display the global system label.

Parameters: None.

Returns: No value; outputs to the console.

6. DisplayInverterTests Function

void DisplayInverterTests(InverterTest *, int, bool = false)

Purpose: Displays all records or warning/fault records using the default argument.

Parameters: array pointer, size and optional fault filter.

Returns: No value; outputs to the console.

7. WriteFaultReport Function

void WriteFaultReport(InverterTest *, int)

Purpose: Exports every warning and fault record to the required alert report.

Parameters: array pointer and number of records.

Returns: No value; writes a text file.

8. DeleteInverterArray Function

void DeleteInverterArray(InverterTest **, int *)

Purpose: Deallocates the dynamic inverter array and resets the pointer and size.

Parameters: array pointer and size pointer.

Returns: No value.

PRINT SCREENS

Main menu:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice:
```

No data loaded:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice: 2

Solar Inverter Diagnostic System - Local diagnostic display
No inverter tests are loaded.

Press any key to continue...
```

Load inverter tests:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice: 1

Input file: inverter_tests.txt
5 inverter tests loaded.

Press any key to continue...
```

Display all diagnostics:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice: 2

Solar Inverter Diagnostic System - Local diagnostic display
ID      DC input    AC output    Efficiency  State
101      5000           4650         93.0       PASS
102      4800.0         4300.0       89.6       WARNING
103      5200.0         0.0          0.0       FAULT (code 17)
104      5100.0         4800.0       94.1       PASS
105      4900.0         4400.0       89.8       WARNING

Press any key to continue...
```

Display warnings and faults:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice: 3

ID      DC input    AC output    Efficiency  State
102      4800         4300         89.6       WARNING
103      5200.0       0.0          0.0       FAULT (code 17)
105      4900.0       4400.0       89.8       WARNING

Press any key to continue...
```

Export fault alerts:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice: 4

3 alert(s) written to inverter_fault_alerts.txt

Press any key to continue...
```

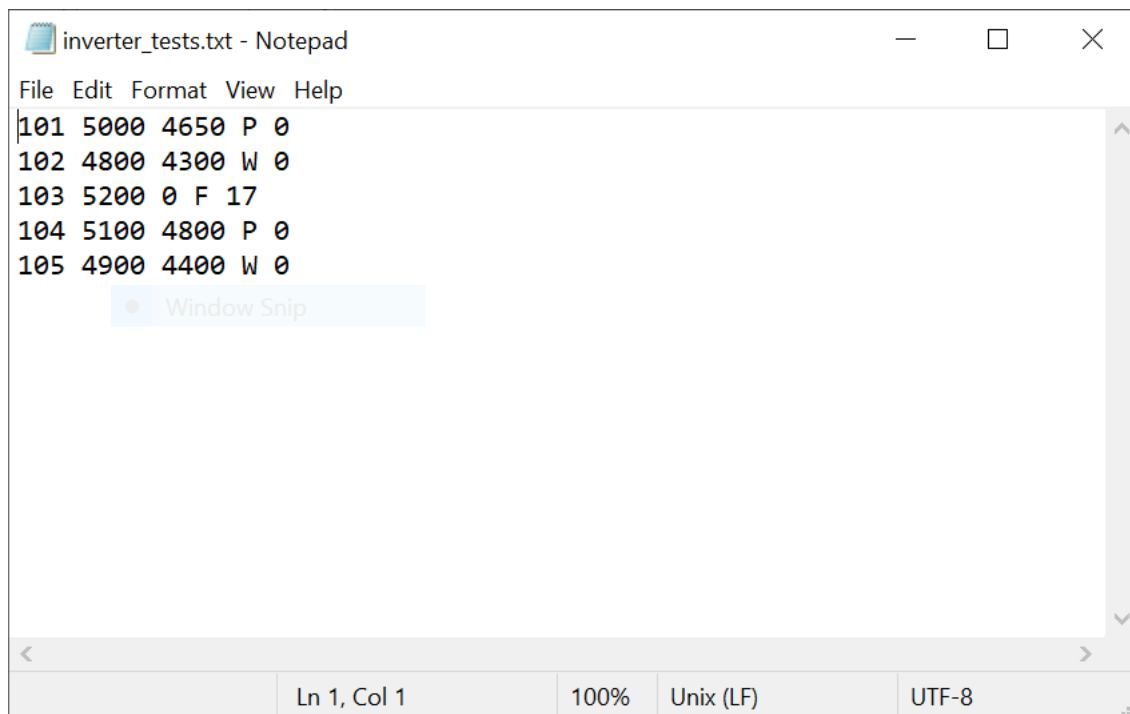
Exit application:

```
Solar Inverter Diagnostic System
ES216BB
Evaluation 1

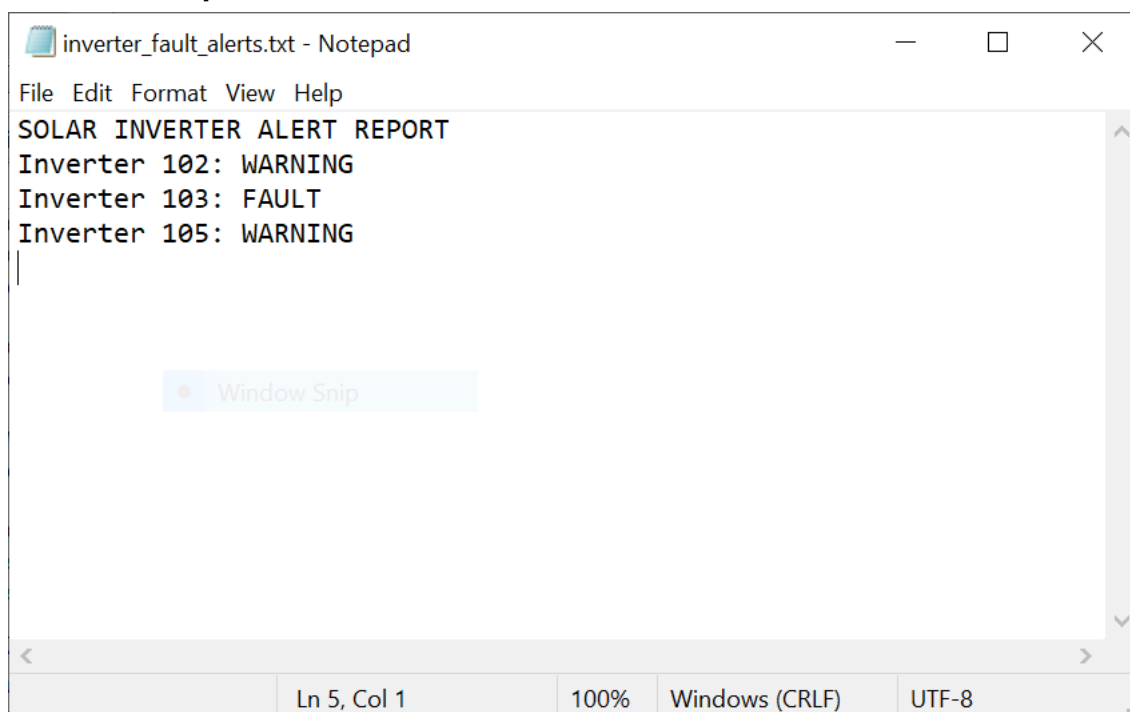
1. Load inverter tests
2. Display all test results
3. Display warnings and faults
4. Export fault alerts
5. Exit
Choice: 5

Diagnostic application closed.

Press any key to continue...
```


Text File Input:

```
inverter_tests.txt - Notepad
File Edit Format View Help
101 5000 4650 P 0
102 4800 4300 W 0
103 5200 0 F 17
104 5100 4800 P 0
105 4900 4400 W 0
Ln 1, Col 1 100% Unix (LF) UTF-8
```

Text File Output:

```
inverter_fault_alerts.txt - Notepad
File Edit Format View Help
SOLAR INVERTER ALERT REPORT
Inverter 102: WARNING
Inverter 103: FAULT
Inverter 105: WARNING
Ln 5, Col 1 100% Windows (CRLF) UTF-8
```

ANNEXURE A – MARK ALLOCATION

Note: Score range is 0 - 4: 0-none, 1-poor, 2-average, 3-good, 4-excellent

TEST RUBRIC	SCORE [0-4]	WEIGHT [%]
C++ CODE EVALUATION		50
0. Structures, types, static and dynamic access		8
1. Function1		6
2. Function2		4
3. Function3		6
4. Function4		5
5. Function5		5
6. Function6		4
7. Function7		3
8. Function8		4
9. Compile and runtime stability		5
TOTAL		50

Graduate Attribute	GA Number	GA Score [0-5]
Application of scientific and engineering knowledge	GA2	0,1,4
Engineering methods, skills, tools, including information technology	GA5	1,2,3,5,6
Impact of Engineering Activity	GA7	4,5,6
Engineering Professionalism	GA10	7,8,9

ANNEXURE B – INFORMATION SHEET

Data types: void, char, short, int, float, double

Data Type modifiers: const, auto, static, unsigned, signed

Arithmetic operators: * / % + -

Relational operators: < <= > >= == !=

Assignment operator: = += -= *= /= %= &= ^= |= <<= >>=

Logic operators: && || !

Bitwise logic operators: & | ^ ~ << >>

Pointer operators: Dereference: * Address: &

Control Structures:

IF Selection: if (condition) { ... };

IF ELSE Selection: if (condition) { ... } else { ... };

WHILE Loop: while (condition) { ... };

DO WHILE loop: do { ... } while (condition);

FOR Loop: for (initial value of control variable; loop condition; increment of control variable) { ... }

SWITCH Selection: switch (control variable){ case 'value': ... ; break; default: ... ; break; }

Functions: return_data_type function_name (parameters) { ... };

Arrays:

One dimensional: data_type variable_name[size];

Two dimensional: data_type variable_name [x_size][y_size];

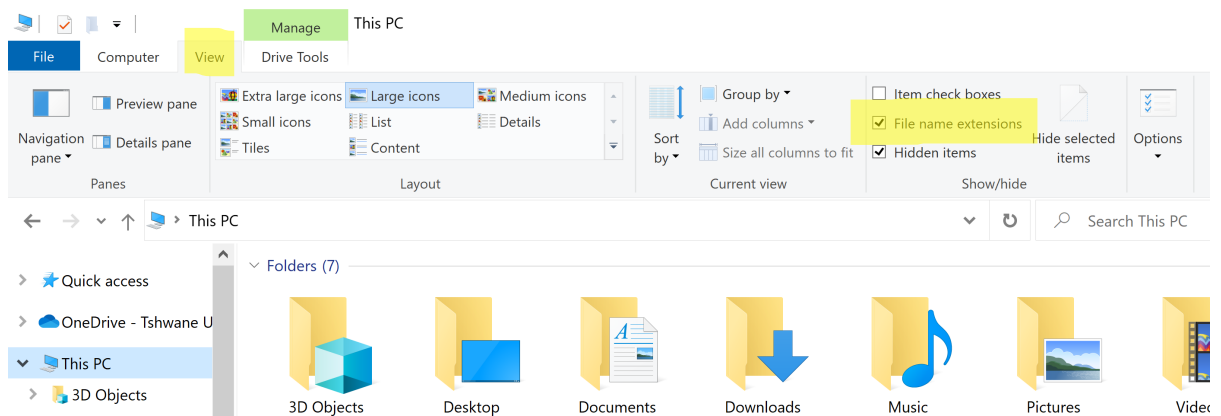
ANNEXURE C – ASCII TABLE

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

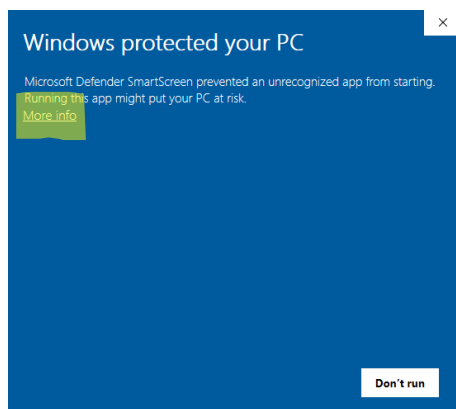
Source: www.LookupTables.com

ANNEXURE D – HOW TO RUN THE SHOWCASE FILE

1. Enable file extensions (see highlighted in yellow)



2. Change the name from **“Showcase.old”** to **“Showcase.exe”**
3. Run the **“ShowcaseEV.exe”** by double-clicking on the icon.
4. Windows may show the following. Click on **“More info”**



5. Click on **“Run anyway”**

